Credit for materials: This course material is largely adapted from the lectures given by Prof. Michalis Vazirgiannis (EP) for the course Machine and Deep Learning.

Introduction to Time Series in Deep Learning

CSC_51054_EP: Deep Learning

Ecole Polytechnique, France

Lecture Instructor: Prof. Ye ZHU

Monge Assistant Professor

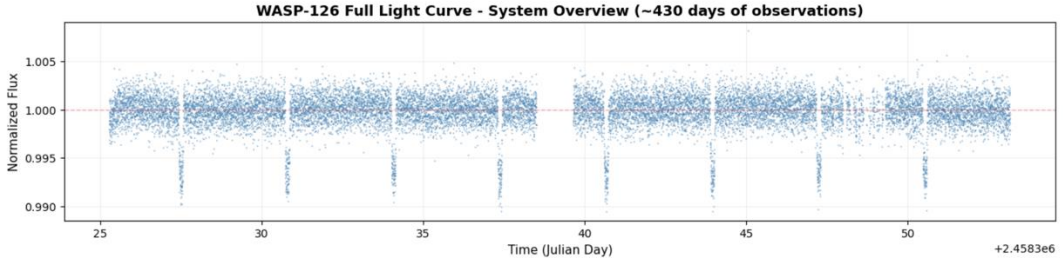
https://l-yezhu.github.io/

November, 2025

Can we predict the future?

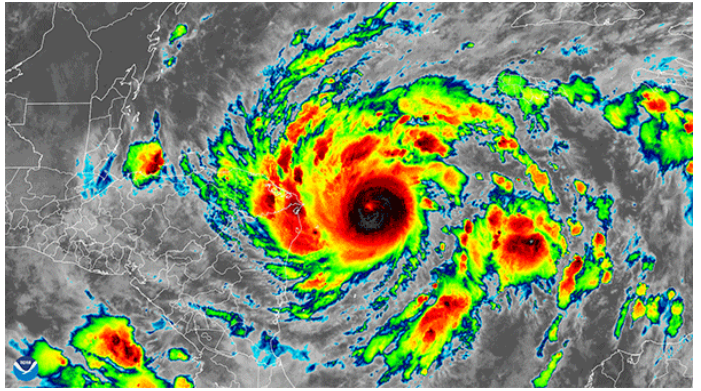


Stock price: hard to predict under random walk hypothesis



TESS: Predictable (periodic, physics-driven)

Challenge: Information decay over time
The further you predict, the worse you get.



Weather: short-term yes, long-term hard

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

Section 3: Deep Learning for Time Series

Important - Break !

Section 4: Discussions

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

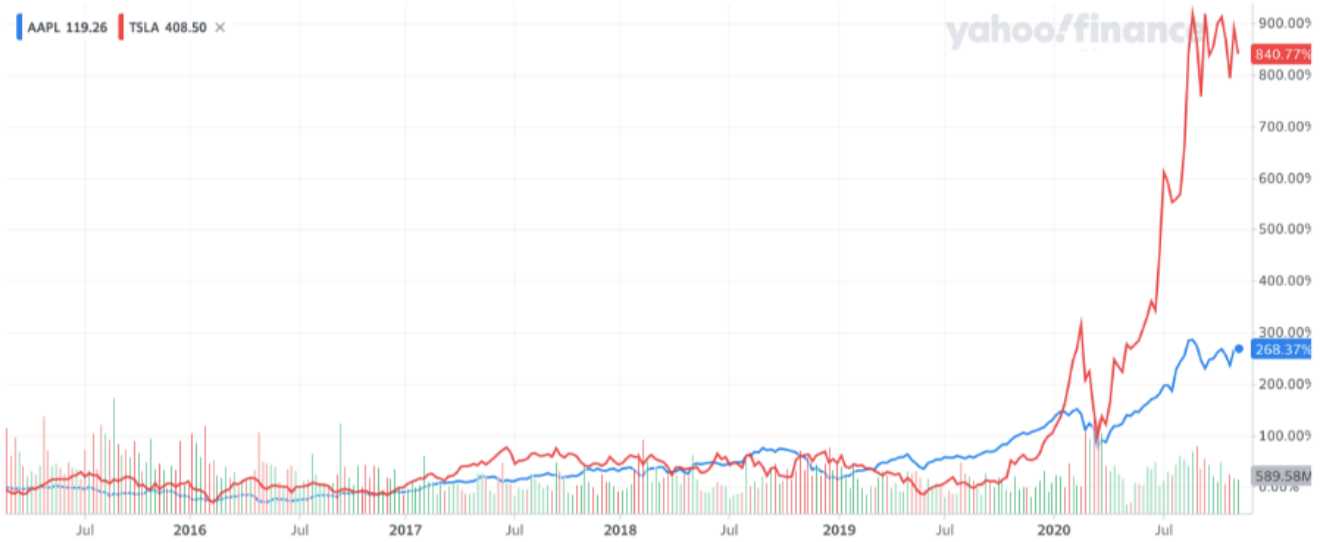
Section 3: Deep Learning for Time Series

Important - Break !

Section 4: Discussions

What are time-series data? – stock prize

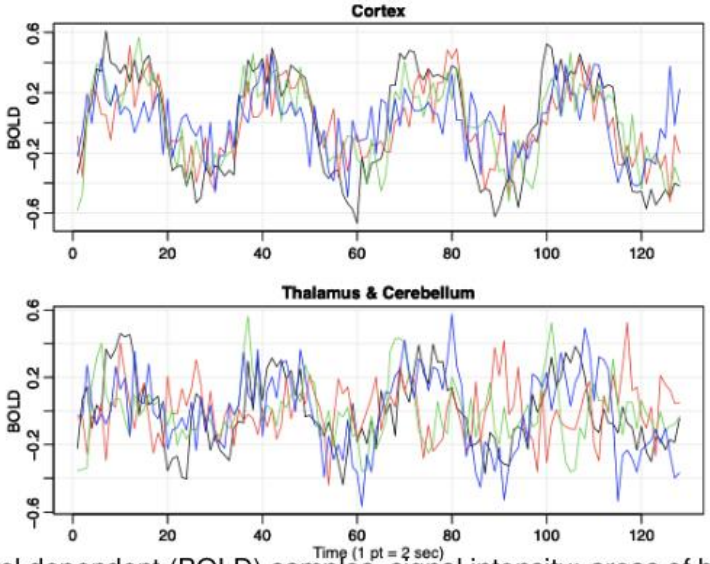
APPLE, TESLA stocks, A 5-year weekly evolution[1]



[1] *source: finance.yahoo.com*

What are time-series data? – fMRI imaging

- fMRI data from various locations in the cortex, thalamus, and cerebellum; n = 128 points, one observation taken every 2 seconds

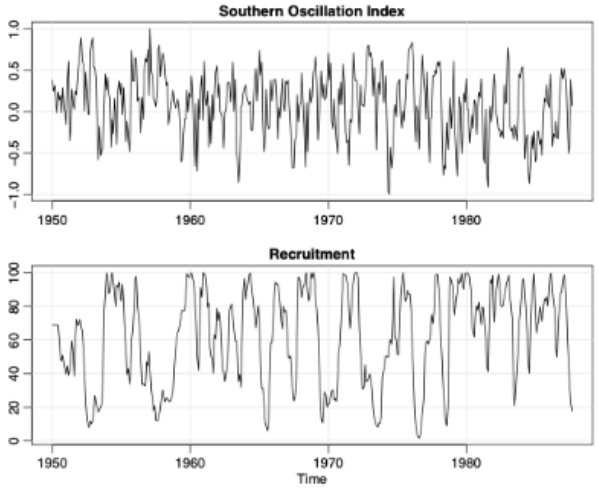


- blood oxygenation-level dependent (BOLD) samples, signal intensity: areas of brain activation in the brain.
- periodicities appear strongly in motor cortex series, less strongly in the thalamus and cerebellum.
- series from different areas of the brain: testing whether the areas responding differently stimulus.
- Analysis of variance techniques: spectral analysis of variance.

[2] source: R. H. Shumway, D. S. Stoffer,, Time Series Analysis and Its Applications, With R Examples. Springer Texts in Statistics

What are time-series data? – weather cycles and fish

Monthly SOI and Recruitment (estimated new fish), 1950-1987 [4]

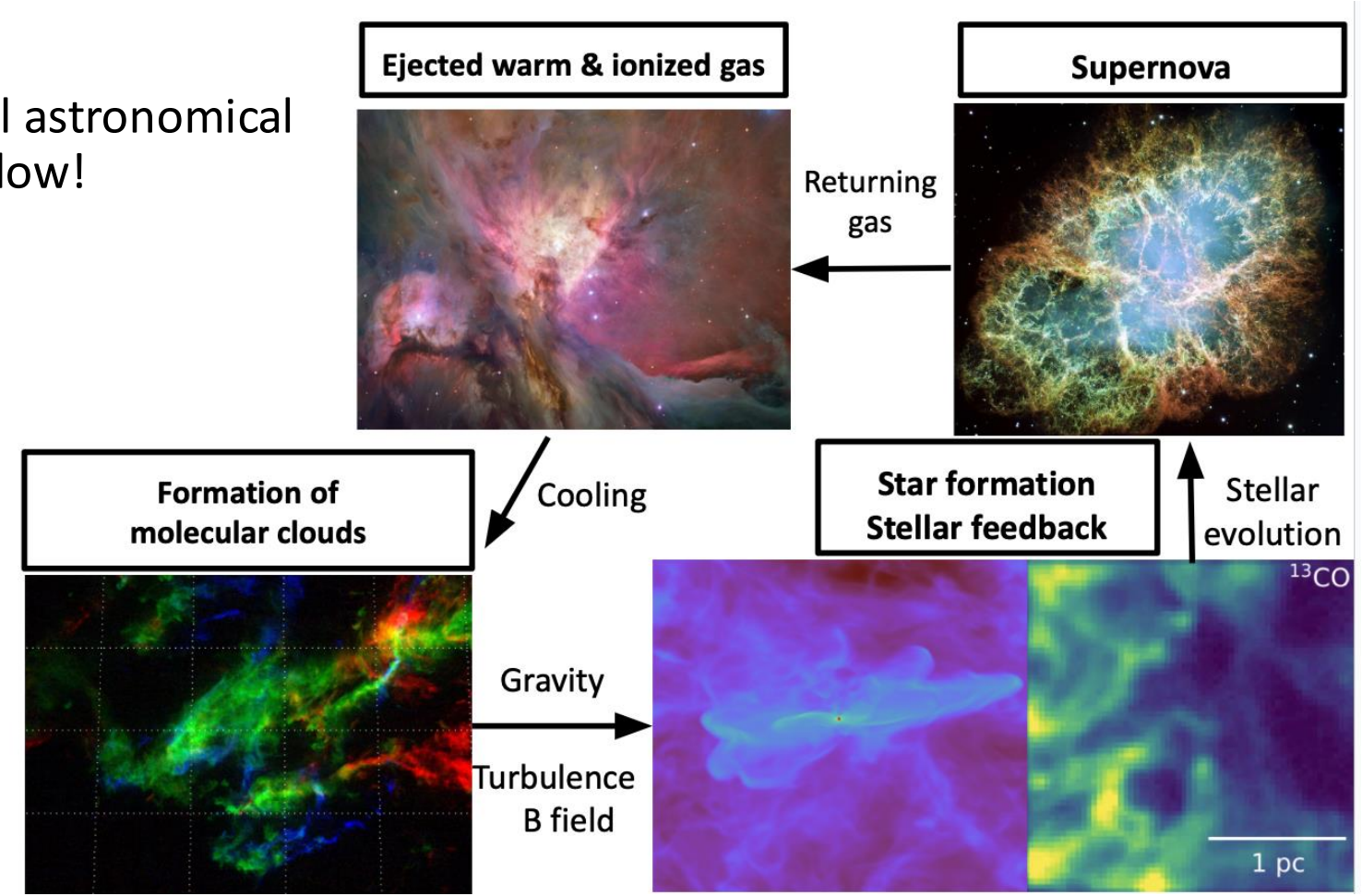


- SOI: changes in air pressure, related to sea surface temperatures - central Pacific Ocean.
- Central Pacific warms every 3-7 years (El Niño effect blamed for global extreme weather).
- Both series exhibit repetitive behavior, (cycles) - underlying processes: rate or frequency of oscillation

[4] R. H. Shumway, D. S. Stoffer, Time Series Analysis and Its Applications, With R Examples. Springer Texts in Statistics

What are time-series data? – astronomical data

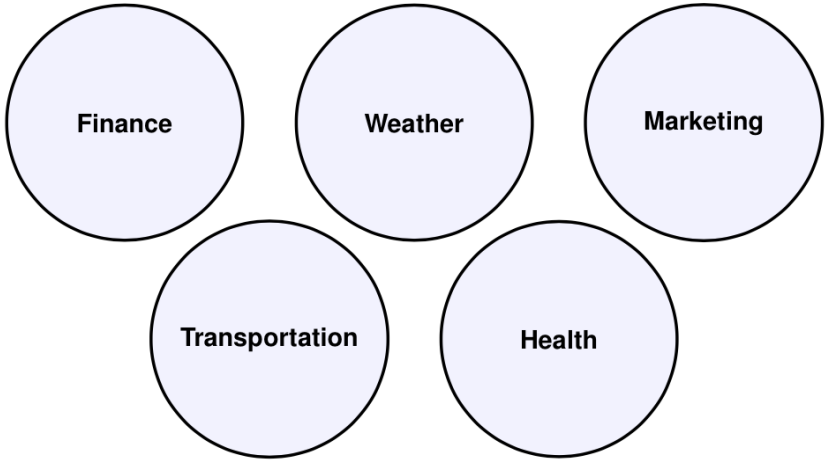
- Lab on real astronomical data to follow!



Motivation for time series modeling

Why focus on Times Series models?

1. **Constant need to predict the future.**
2. **Focus on minimizing risk, developing long-term strategies, making decisions instantly.**
3. **Availability of a great amount of data.**
4. **Impact on a huge range of applications.**

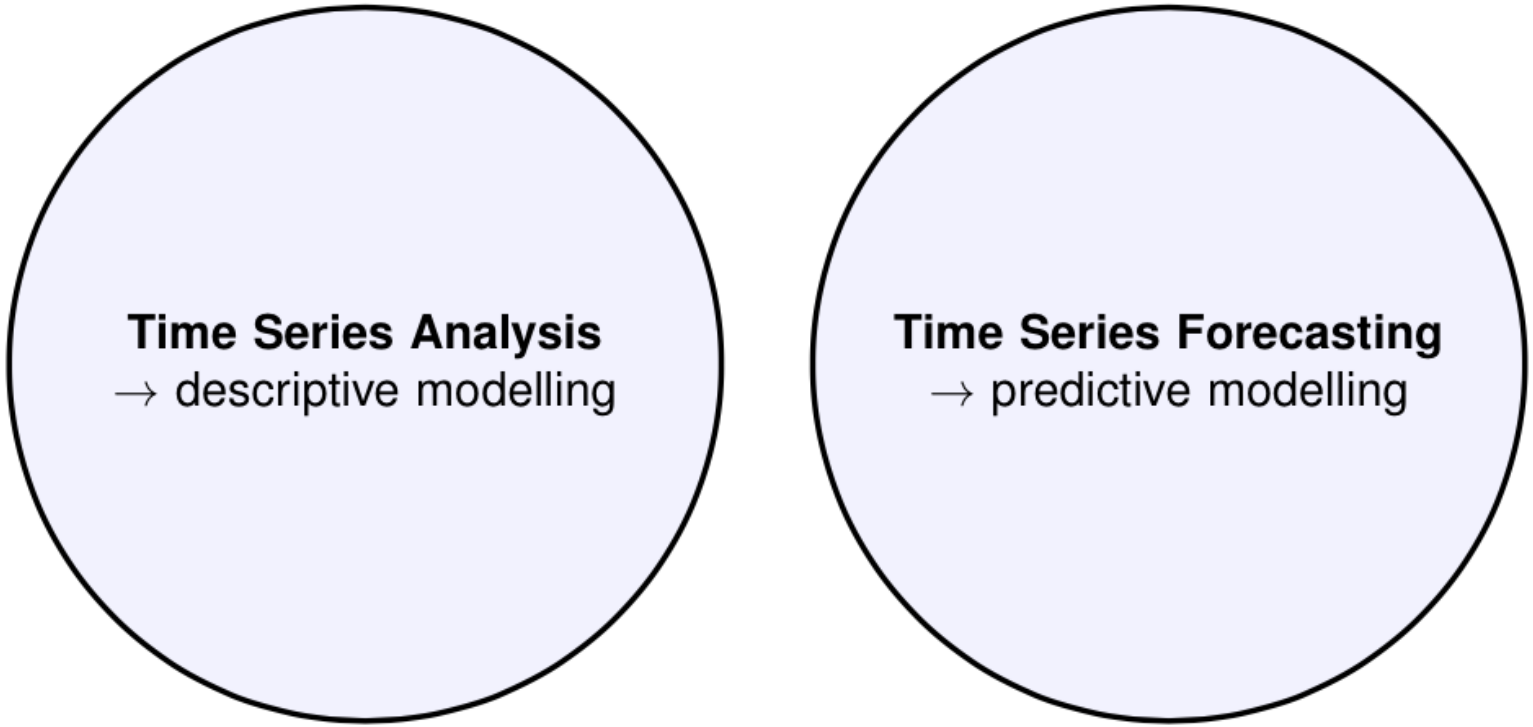


Applications - examples

Forecasting the

- new cases of covid-19 in a city each day.
- closing price of a stock each minute.
- product sales in units sold each day for a store.
- birth rate at all hospitals in a city each year.
- utilization demand on a server each hour.
- average price of gasoline in a city each day.
- number of passengers through a train station each day.

Domains of study



Task 1: Time series analysis and classification -> lab

- developing *models* that best capture or describe an observed time series in order **to understand the underlying causes**. Seek the **"why"***model* behind a time series dataset.
- Necessary assumptions about the form of the data and decomposing the time series into constituting components.
- Quality of a descriptive model: i. how well it describes all available data ii. the quality of the interpretation it provides for the problem domain.

"The primary objective of time series analysis is to develop mathematical models that provide plausible descriptions from sample data." [5]

[5] Time Series Analysis: With Applications in R, (Springer Texts in Statistics) 2nd Edition, Cryer, Chan

Task 2: Time series forecasting -> lab

- Making *predictions* about the future: called extrapolation in the classical statistical handling of time series data.
- More modern fields focus on the topic and refer to it as time series forecasting.
- Forecasting: *models to fit on historical data and using them to predict future observations*.
- Descriptive models cannot predict future (i.e. to smooth or remove noise), they only seek to best describe the data.
- In forecasting future is not known and must only be estimated from the past samples.

"The purpose of time series forecasting is generally two-fold:
1. model the stochastic mechanisms that generate the observed series
2. predict forecast the future values of a series based on the history of series values." [6]

[6] Time Series Analysis: With Applications in R, (Springer Texts in Statistics) 2nd Edition, Cryer, Chan

Other tasks for time series

- Anomaly detection
  - Identify anomalous patterns or outliers in time series data by detecting deviations from normal behavior—crucial for detecting supernovae, strong gravitational lensing, or instrument malfunctions in astronomical surveys.

- Imputation
  - Recover missing values in incomplete time series caused by observation gaps, sensor failures, or data corruption—essential for continuous monitoring and downstream analysis.

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

- **AR / ARMA / ARIMA**
- **Manual features & distance similarity**

Section 3: Deep Learning for Time Series

Important - Break !

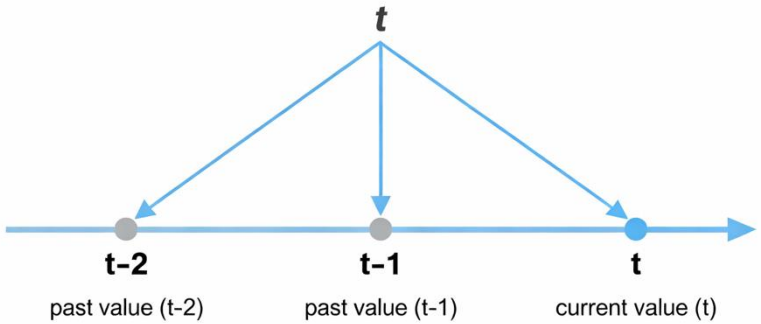
Section 4: Discussions

Classic approaches: statistics & simple ML

- AR (autoregressive)/ARMA (autoregressive and moving average)/ARIMA (autoregressive integrated moving average) models:

  Use past values to linearly predict future points.

  - Pros: Fast, interpretable, reliable with stationary data.
  - Cons: Can't handle nonlinear or nonstationary patterns well.



Details about AR, ARMA and ARMIA

AR (Autoregressive):

- Predicts future values from a linear combination of its own previous values.
- Good for stationary signals—trends must be minimal or removed.

MA (Moving Average):

- Represents each point as a linear combination of past random shock (noise) terms.
- Captures short-term ‘blips’ or randomness that repeats.

ARMA:

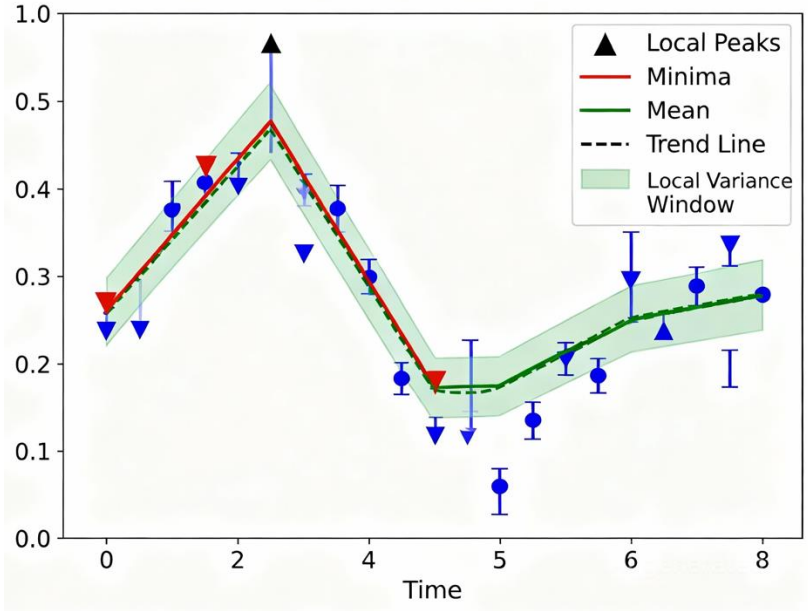
- Combines AR and MA for stationary time series.
- Handles both persistence (AR) and noise (MA).

ARIMA (Integrated):

- Integrated/differencing to model trends and non-stationarity.
- Most general: ARIMA can handle trends, cycles, and noise, widely used in real applications.

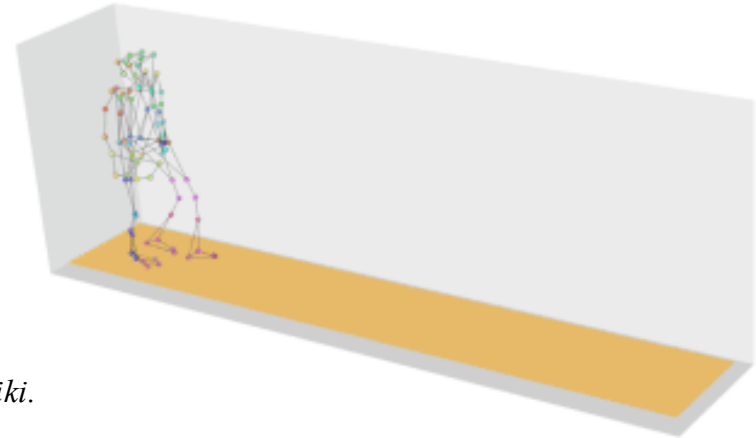
From manual features to ML

- Manual features: Mean, std, min, max, trend, frequency peaks
- Shallow ML: Classify or regress using hand-crafted features (e.g., logistic regression, SVM, random forest)



Measuring similarity: Euclidean vs. DTW (lab to follow)

- Euclidean Distance: Compares time series shape point by point. Fails if there are small time shifts.
- Dynamic Time Warping (DTW): Flexibly aligns time series in time and matches “shape” even with misalignments.
- In practice: Many scientific time series (e.g., exoplanet transit data in our lab) are better classified by comparing “shape” rather than specific values at every timestamp.



Gif credits to DTW wiki.

Details of DTW - Measuring Similarity Between Time Series

1. Distance Matrix Construction: Compute pairwise distances between points in two sequences, forming a matrix where each element holds the distance between corresponding time points.
2. Accumulated Cost Matrix: Use dynamic programming to build a matrix of the minimum cumulative distance to each point, allowing flexible alignments (warping) in the time dimension.
3. Optimal Warping Path Finding: Backtrack from the bottom-right to top-left of the cost matrix to find the path with minimum total distance that aligns the two sequences.
4. DTW Distance: The total cost along this optimal path is the DTW distance.

```
int DTWDistance(s: array [1..n], t: array [1..m]) {
    DTW := array [0..n, 0..m]

    for i := 0 to n
        for j := 0 to m
            DTW[i, j] := infinity
    DTW[0, 0] := 0

    for i := 1 to n
        for j := 1 to m
            cost := d(s[i], t[j])
            DTW[i, j] := cost + minimum(DTW[i-1, j  ],    // insertion
                                        DTW[i  , j-1],    // deletion
                                        DTW[i-1, j-1])    // match

    return DTW[n, m]
}
```

A recap of statistical methods

Limitations of Statistical Methods

Focus on

- complete data
- linear relationships
- fixed temporal dependencies
- univariate data
- 1-step forecasts

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

Section 3: Deep Learning for Time Series
- Basic components

Important - Break !

Section 4: Discussions

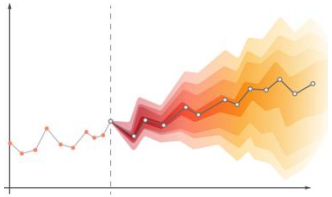
Why deep learning for time series?

Why use Deep Learning for Time Series?

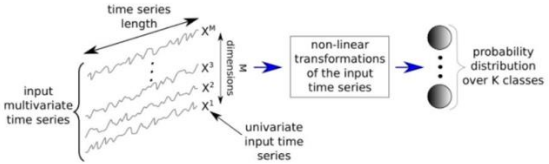
- Learns patterns in data with complex non-linear dependencies
- Supports multiple inputs and outputs
- Has shown good performance in many scenarios
- Models are flexible and expressive
- Can be trained with large datasets
- Easy to introduce exogenous features into the model

Different tasks in time series

Forecasting
(point or probabilistic)



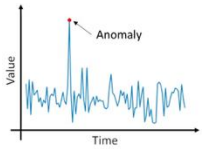
Classification
(to identify a time series as coming from one of possibly many sources or predefined groups)



Other tasks:

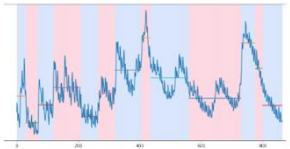
1. Anomaly Detection

Find abnormal events in a time series



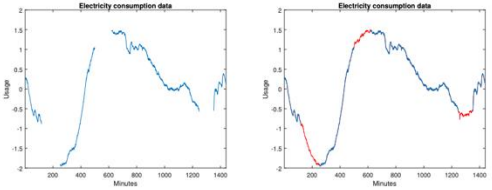
2. Segmentation/ change-point detection

Find significant abrupt changes in the time series



3. Completion/interpolation

Recover missing/lost samples in a time series



4. Query by content/indexation

Given an input time series, retrieve the closest time series in a large database up to a given measure of fit

DL architectures for time series forecasting

1-step-ahead forecasting:

$$\hat{y}_{i,t+1} = f(y_{i,t-k:t}, \boldsymbol{x}_{i,t-k:t}, \boldsymbol{s}_i)$$

- $\hat{y}_{i,t+1}$: Model forecast
- $y_{i,t-k:t}$: observations of target over a look-back window k
- $\boldsymbol{x}_{i,t-k:t}$: external inputs over a look-back window k
- $\boldsymbol{s}_i$: static metadata associated with the entity (e.g. sensor location)

multi-horizon forecasting:

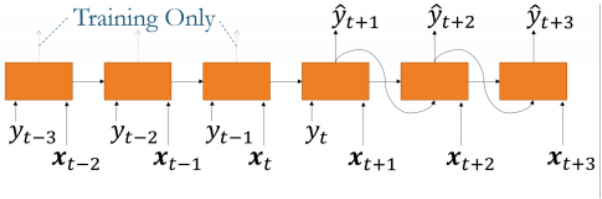
$$\hat{y}_{t+\tau} = f(y_{t-k:t}, \boldsymbol{x}_{t-k:t}, \boldsymbol{u}_{t-k:t+\tau}, \boldsymbol{s}, \tau)$$

- $\hat{y}_{t+\tau}$: τ is a discrete forecast horizon
- $\boldsymbol{u}_{t-k:t+\tau}$: known future inputs (e.g. date information, such as the day-of-week or month) across the entire horizon

Time Series Forecasting With Deep Learning: A Survey, Bryan Lim and Stefan Zohren, https://arxiv.org/pdf/2004.13408.pdf

Basic building blocks

Multi-horizon Forecasting Models



(a) Iterative Methods



(b) Direct Methods

autoregressive deep learning architectures, produce multi-horizon forecasts by recursively feeding samples of the target into future time steps

→ large error accumulations

produce forecasts directly using all available inputs, make use of sequence-to-sequence architectures, (an encoder summarizes past information and a decoder combines them with known future inputs)

→ Fixed maximum forecast horizon

Output and loss functions

1. **Point Estimates:** determine the expected value of a future target: reformulate problem to a classification task for discrete outputs (e.g. forecasting future events), and regression task for continuous outputs.

$$\mathcal{L}_{classification} = -\frac{1}{T}\sum_{t=1}^{T} y_t \log(\hat{y}_t) + (1 - y_t)\log(1 - \hat{y}_t)$$

$$\mathcal{L}_{regression} = \frac{1}{T}\sum_{t=1}^{T} (y_t - \hat{y}_t)^2$$

2. **Probabilistic Outputs:** model uncertainties, use deep neural networks to generate parameters of known distributions. e.g. Gaussian to forecast continuous targets

   networks output: mean, variance parameters for the predictive distributions / step:

$$y_{t+\tau} \sim N(\mu(t,\tau), \zeta(t,\tau)^2),$$

$$\mu(t,\tau) = \boldsymbol{W}_\mu \boldsymbol{h}_t^L + \boldsymbol{b}_\mu,$$

$$\zeta(t,\tau) = \text{softplus}(\boldsymbol{W}_\Sigma \boldsymbol{h}_t^L + \boldsymbol{b}_\Sigma),$$

   softplus activation: ensure variance takes only positive values

Time Series Forecasting With Deep Learning: A Survey, Bryan Lim and Stefan Zohren, https://arxiv.org/pdf/2004.13408.pdf

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

Section 3: Deep Learning for Time Series
- Basic components

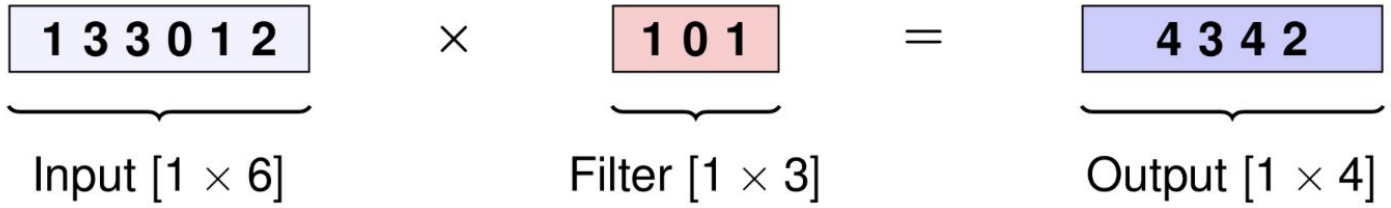
Important - Break !

Section 4: Discussions

CNN (Convolutional Neural Networks)

- Convolution layer
  - units in the hidden layer operate on a field of the output
  - weights are shared across input
- Motivation for CNNs in sequence learning
  - Success of CNN in CV and recently in NLP
  - Achieves state-of-the-art accuracy in audio synthesis and machine translation
- Main advantages:
  - Lower level of model complexity than RNNs
  - Parallelization of computations
  - Dilated CNN can outperform RNN in sequence modelling
- 2D Convolutions encode spatial invariance
- 1D Convolutions encode temporal invariance, “stationarity”

1D Convolutions



1D Convolution with 1 filter ($stride = 1$), $t = 6$, $\#features = 1$

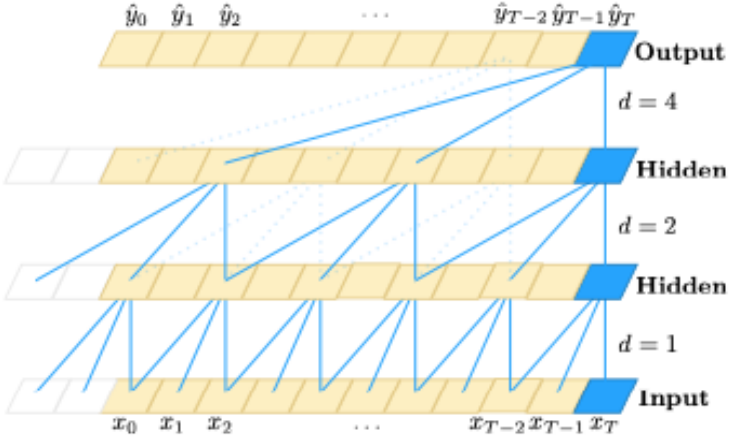
- A convolution kernel that is convolved with the input over a single temporal dimension
- Filters are trained to detect features in the sequence, regardless of where they appear
- Multiple filters can detect multiple features
- Can be implemented to multivariate time series (Input [$t \times \#features$])

Causality

- *Causal constraint*: y_t depends only on $x_0,...,x_t$ and not on any "future" inputs $x_{t+1},...,x_T$.
- Learning: network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0,...,y_T,f(x_0,...,x_T))$
- This formalism encompasses settings
  - auto-regressive prediction: predict some signal given its past:
    - setting the target output to be simply the input shifted by one time step.
  - does not capture domains (machine translation, or sequence- to-sequence prediction)
    - the entire input sequence (including *"future" states*) can be used to predict each output

Causal convolution

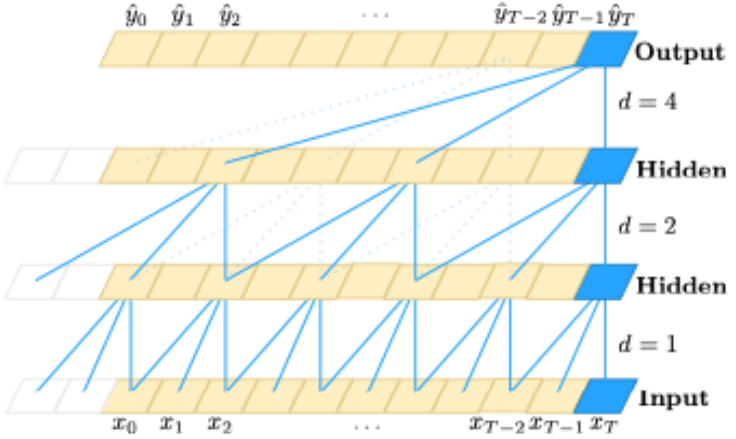
- Temporal Convolution Networks (TCN) constraints:
  - network produces an output of the same length as the input
    - To accomplish this: uses a 1D fully-convolutional network (FCN) architecture
    - each hidden layer is the same length as the input layer, and zero padding of length (kernel size − 1) is added to keep subsequent layers the same length as previous ones.
  - there can be no leakage from the future into the past.
    - TCN uses *causal convolutions:* output at time t is convolved **only** with elements <t in the previous layer.
  - *TCN = 1D FCN + causal convolutions*



An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, Shaojie Bai. J. Zico Kolter, Vladlen Koltun, https://arxiv.org/pdf/1803.01271.pdf

Causal convolution

- Disadvantage: in order to achieve a long effective history size need large kernel size - extremely deep network or very large filters
- *Dilated convolution*: to allow for both very deep networks and very long effective history



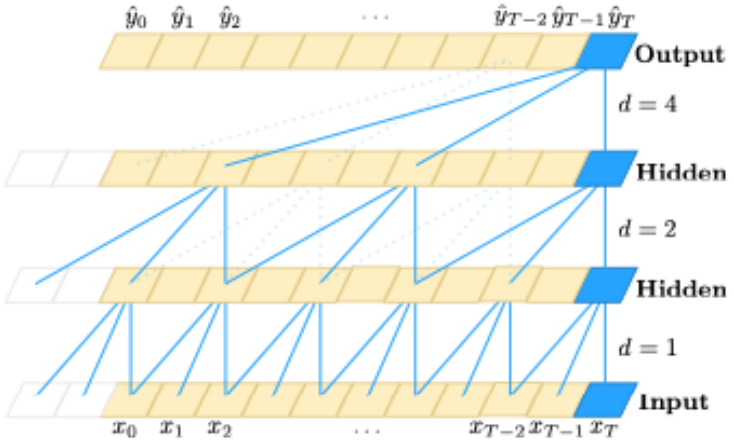
An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, Shaojie Bai. J. Zico Kolter, Vladlen Koltun, https://arxiv.org/pdf/1803.01271.pdf

Dilated convolution

- 1-D sequence input $x \in R^n$ and a filter $f:\{0,...,k-1\} \rightarrow R$,
- Dilated convolution operation F on element s of the sequence:

$$F(s) = (\mathbf{x} *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i}$$

- *d*: dilation factor, k: filter size, $(s - d \cdot i)$ direction of the past.
- is equivalent to introducing a fixed step between every two adjacent filters.
- d = 1: regular convolution
- d>1: enables output at the top level to represent a wider range of inputs: expanding the receptive field of a ConvNet.

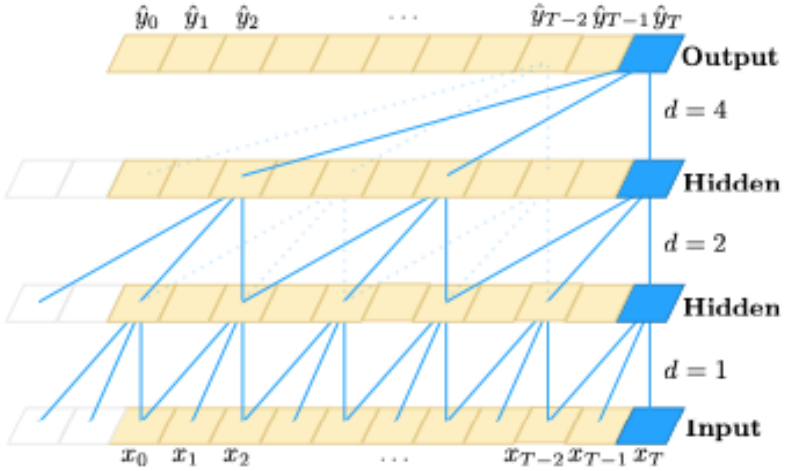


An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, Shaojie Bai. J. Zico Kolter, Vladlen Koltun, https://arxiv.org/pdf/1803.01271.pdf

Dilated convolution

Longer memory - increase the receptive field of the TCN:
- choosing larger filter sizes *k*
- increasing the dilation factor *d*:
- effective history for this layer *(k − 1)d*.

In dilated convolutions, increase d exponentially with the depth of the network: $d = O(2^i)$. *i:* level in the network. Ensures
- each input treated by some filter in the effective history,
- allowing for an extremely large effective history using deep networks.



An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling, Shaojie Bai. J. Zico Kolter, Vladlen Koltun, https://arxiv.org/pdf/1803.01271.pdf

Language modeling & time series – connections

- At their core, both language modeling and time series prediction are about learning to predict “what comes next” in a sequence.
    - RNNs, LSTMs, and Transformers were all first developed for sequential prediction—whether that’s the next word or the next measurement.
    - In language: input is a sequence of words or tokens.
    - In time series: input is a sequence of numerical values.

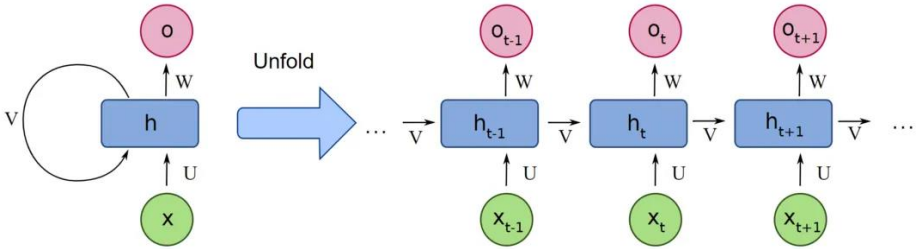
Models in NLP vs. time series: key similarities and differences

Model	Common NLP use case	Time series use case	Key differences
RNN	Character or word-level LM	Univariate/multivariate TS reg/pred	Both capture short-range dependencies, but RNNs often struggle with long memory in both domains.
LSTM	Translation, sequence tagging	Multi-step forecasting, event detection	LSTM's gating makes long dependencies tractable in both, but TS often lower dimension/less vocab.
Transformer	BERT, GPT, LLMs	TS forecasting, anomaly detection	Self-attention = global context in both, but language has rich discrete structure; TS often requires position encoding for numeric continuity.

RNN (Recurrent Neural Network)

- RNNs model the order of data explicitly.
- The chain-like nature reveals that RNNs are intimately related to sequences.
- Used to be the most common choice for sequence modeling.

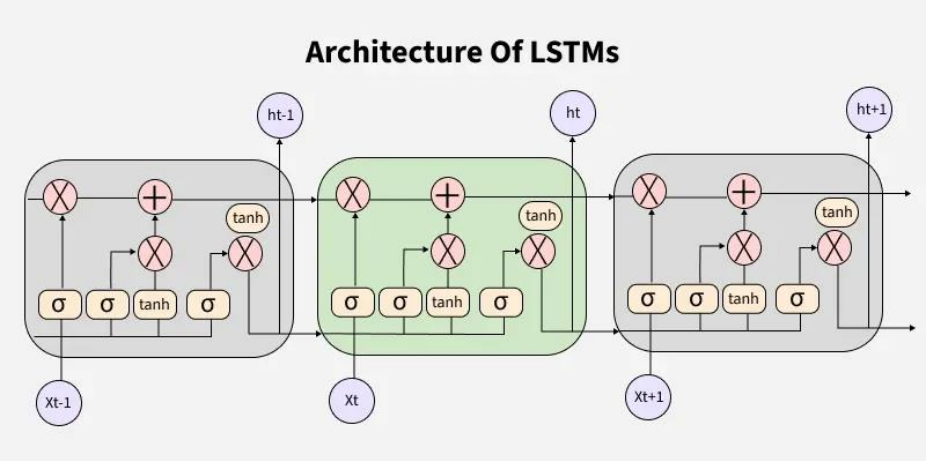
$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = g(W_{hy}h_t + b_y)$$



Limitation: Backpropagated gradients exponentially shrink or grow through many steps, making long-range dependencies hard to learn.

LSTM (Long Short-Term Memory Network)

✓ Handles long-term dependencies
✓ Less feature engineering
✗ Slow (sequential compute)
✗ Hard to parallelize

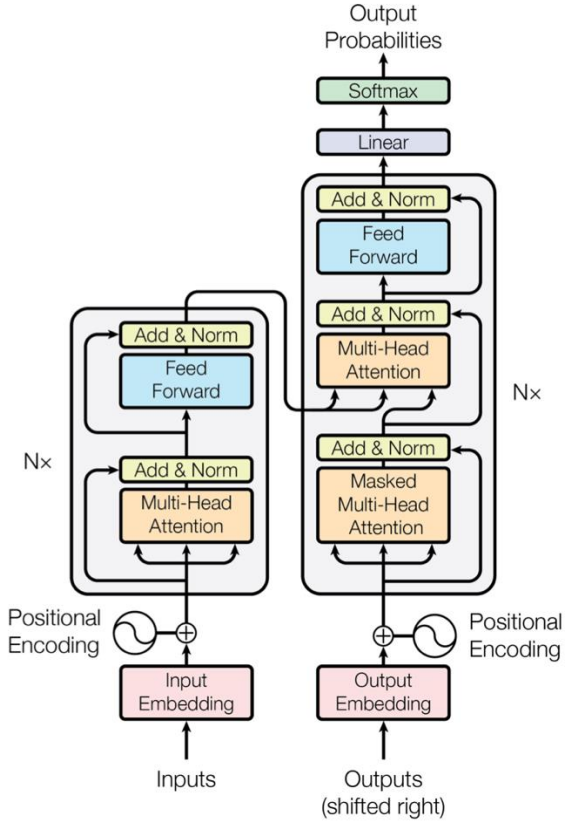


$$
\begin{aligned}
f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\
i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t] + b_c) \\
o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

Transformer - Attention-based models

✓ Fully parallel computation
✓ Learns arbitrary range dependencies
✓ Transparent via attention heatmaps

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Hybrid models for time series forecasting

Hybrid models (combinations of different achitectures) are usually used to detect the complex patterns of time-series:

- combinations of CNN and RNN layers are very frequent
- combinations of ARIMA and ANNs have also been explored
- modern modules (such as Autoencoders and Attention layers) are recently combined with RNNs

Agenda

Section 1: Introduction to Time Series

Section 2: Classic Approaches: Statistics & Simple ML

Section 3: Deep Learning for Time Series
- Basic components

Important - Break !

Section 4: Discussions

Is DL the optimal solution?

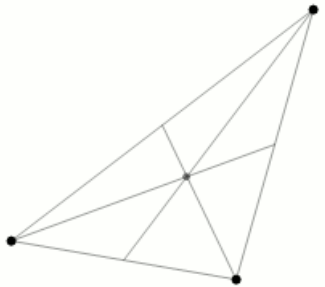
DL approaches are flexible and achieve state-of-the-art results for several tasks BUT…

- Good performances does NOT mean good understanding of the data/good results
- DL when used as a black box (without understanding of the data/their properties/models) is not enough, since experts cannot interpret the results

Can you think of examples where DL may fail?

- Deep learning excels when the future is structured and compressible (periodicity, regularity, or low randomness).
- When the future is fundamentally unpredictable due to chaos or high complexity, no data-driven method can 'learn' its way out—understanding the physical law is essential.

Then what can we possibly do to improve the current AI systems?



Gif credits to wiki.

Conclusions

- Deep learning models (RNN, LSTM, Transformer) excel at learning complex temporal patterns in structured time series data, often surpassing classical methods.
- Their effectiveness depends on data quality, quantity, and the inherent predictability of the system; periodic and stable signals are easier to model than chaotic or highly complex ones.
- Deep learning remains fundamentally data-driven, lacking explicit understanding of underlying physical laws, which limits generalization to unseen regimes or unpredictable phenomena.
- For practical applications, combine classical insights, deep learning advances, and domain expertise thoughtfully.